Unit → 2

→ Decimal to Binary

Q 2026 · 625

2	2026	0
2	1013	1
2	506	0
2	253	1
2	126	0
2	63	1
2	31	1
2	15	1
2	7	1
2	3	1
	1	

$0.625 \times 2 = 1.25 \quad 1$

$0.25 \times 2 = 0.50 \quad 0$

$0.50 \times 2 = 1 \quad 1$

$(11111101010.101)_2$

→ Decimal to octal

2026 · 625

8	2026	2
8	253	5
8	31	7
	3	

$0.625 \times 8 = 5.0$

$= (3752.5)_8$

10	11	12	13	14	15
A	B	C	D	E	F

→ Decimal to Hexadecimal

2026 · 625

16	2026	10 (A)
16	126	14 (E)
	7	

$0.625 \times 16 = 10.0$

$= A$

$= (7EA \cdot A)_{16}$

→ Binary to decimal

$1011001 \cdot 01001$

$2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0 \quad 2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}\ 2^{-5}$

$1X2^6 + 1X2^4 + 1X2^3 + 1X2^0 + 1X2^{-2} + 1X2^{-5}$

$64 + 16 + 8 + 1 + \frac{1}{4} + \frac{1}{32}$

$89 + \frac{9}{32} = 89 + 0.28125$

$= 89.28125$

→ Binary to octal

Make pairs of 3

$\underset{5}{101}\ \underset{5}{101}\ \underset{3}{011}\ \underset{6}{110} \cdot \underset{5}{101}\ \underset{5}{101}$

5536.55

→ Binary to Hexadecimal

$1011\ 0101\ 1110 \cdot 1011\ 0100$

Make pair of 4

11 5 14 11 4

B E B

$B5E \cdot B4$

→ Octal to Decimal

$(6537.24)_8$

$6 \times 8^3 + 5 \times 8^2 + 3 \times 8 + 7 + 2 \times \frac{1}{8} + 4 \times \frac{1}{64}$

$6 \times 512 + 5 \times 64 + 24 + 7 + 0.25 + 0.625$

3423.3125

→ Octal to binary

6537.24

Replace each bit by group of 3 equivalent binary bits

6 = 110	3 = 011	2 = 010
5 = 101	7 = 111	4 = 100

110 101 011 111 . 010 100

→ Octal to Hexadecimal

6537.24

Replace by 4 binary bits

6 = 0110	3 = 0011	2 = 0010
5 = 0101	7 = 0111	4 = 0100

0110 0101 0011 0111 . 0010 0100

→ Hexadecimal to Decimal

3AF . C6

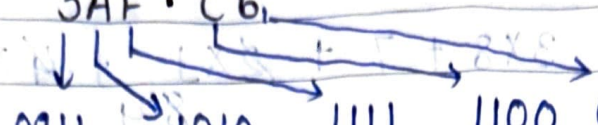
$3 \times 16^2 + 10 \times 16 + 15 + 12 \times 16^{-1} + 6 \times 16^{-2}$

(A) (C)

$= 943.77734375$

→ Hexadecimal to Binary

3AF · C6



0011 1010 1111 1100 0110

0011 1010 1111 · 1100 0110

→ Hexadecimal to octal

3AF · C6

Convert to Binary & then octal

0011 1010 1111 · 1100 0110

1 6 5 7 6 1 4

1657 · 614

→ Complement

- It's used to transform a number wrt it's base
- r's complement & (r-1)'s complement

r → radix / base

r	r's comp.	(r-1)'s comp.
2	2's	1's
8	8's	7's
10	10's	9's
16	16's	15's

r's complement = $r^n - N$

r → base

n → no of digits in N

N → no whose r's comp is to be found

$(r-1)$'s complement $= (r^n - 1) - N$

Also,
r's comp = $(r-1)$'s comp + 1

→ Fixed point Representation
Fixed number of bits are there before and after radix / binary / decimal point (•)

Integer	Fraction

↳ unsigned

sign	Integer	Fraction

↳ signed

• Representation of signed numbers

1. Sign magnitude



1 → -ve number
0 → +ve number

eg → +7 −9

0000 0111 1000 1001

+ 7 − 9

2. 1's compliment
write signed magnitude representation & then invert all bits

eg → +9 −7

00001001 10000111

= 11110110 = 01111000

3. 2's complement

→ Invert all bits & add 1

eg →

+19

signed → 00010011

1's → 11101100

2's → 11101101

→ Advantages of fixed point representation

- simple, fast
- cheap, low power
- Fixed, pricise (upto limits)

→ Disadvantages

- Limited range → easy overflow
- limited pricision
- can't represent very large & very small values together

→ Floating Point Representation

- It has two diffrent parts i.e. integer and fraction
- Also called real number
- scientific notation is used for such representation :-

$M \times B^e$ → exponent

↓ Mantissa

↓ Base

eg → floating point of 163.75

1. Find binary equivalent

2	163	1
2	81	1
2	40	0
2	20	0
2	10	0
2	5	1
2	2	0
	1	

$0.72 \times 2 = 1.54 \quad 1$

$0.54 \times 2 = 1.08 \quad 1$

$0.08 \times 2 = 0.16 \quad 0$

$0.16 \times 2 = 0.32 \quad 0$

$0.32 \times 2 = 0.64 \quad 0$

$0.64 \times 2 = 1.28 \quad 1$

≈ 10100011.11

2. Normalise

In decimal system,

$163.75 = 1.6375 \times 10^2$

11 ly

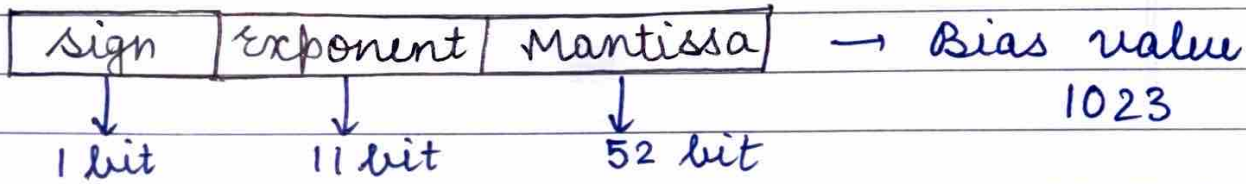
$10100011.11 = 1.01000111 \times 2^7$

3. Representation (IEEE 754 formats)

> 32 Bit Representation (Singe Precision)



64 Bit Representation (Double Precision)



Now 163.75

$= 1.01000111 \times 2^7$

Now sign bit = 0 (+ve no)

exponent = 7

But we add bias value to power so we can get both -ve nos & +ve

$\Rightarrow e = 127 + 7 = 134$

↳ Bias value (32 bit rep)

2	134	0
2	67	1
2	33	1
2	16	0
2	8	0
2	4	0
2	2	0
	1	

$\Rightarrow e = 10000110$

mantissa = 01000111

Representation :-

0	10000110	01000111 00000000000

14 zeros to make 23 bits

→

Exponent	Mantissa	Meaning
all 0's	0	±0
all 0's	≠0	Denormalized number
all 1's	0	±∞
all 1's	≠0	Not a number

→ Addition

```
  101100
+ 110001
---------
 1011101
```

A	B	S	C
1	0	1	0
0	0	0	0
0	1	1	0
1	1	0	1

→ subtraction using 1's complement

−25 − 16

25 = 00011001

1's → 11100110

16 = 00010000

1's → 11101111

Add

```
  11100110
+ 11101111
----------
 111010101
```

add carry from MSB addition to LSB (end around carry)

```
 11010101
        +1
----------
 11010110
```

$2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

MSB = 1 (Ans → −ve)

take 1's comp. to get magnitude

00101001

↓ ↓ ↓

2^5 2^3 2^0

= 32+8+1 = 41

also −ve ⇒ −41

→ Subtraction using 2's complement

$-25 - 16$

$25 = 00011001$	$16 \rightarrow 00010000$
$1's \rightarrow 11100110$	$1's \rightarrow 11101111$
$2's \rightarrow 11100111$	$2's \rightarrow 11110000$

Add $11100111 + 11110000 = 1\,11010111$

↓ Discard carry

$= 11010111$

↓ −ve no.

$10101111 = 41$

$\Rightarrow -41$

→ Overflow condition

(+) + (+) = (−)] → overflow

(−) + (−) = (+)]

Checked by

Cin to sign bit ⊕ Cout from sign bit

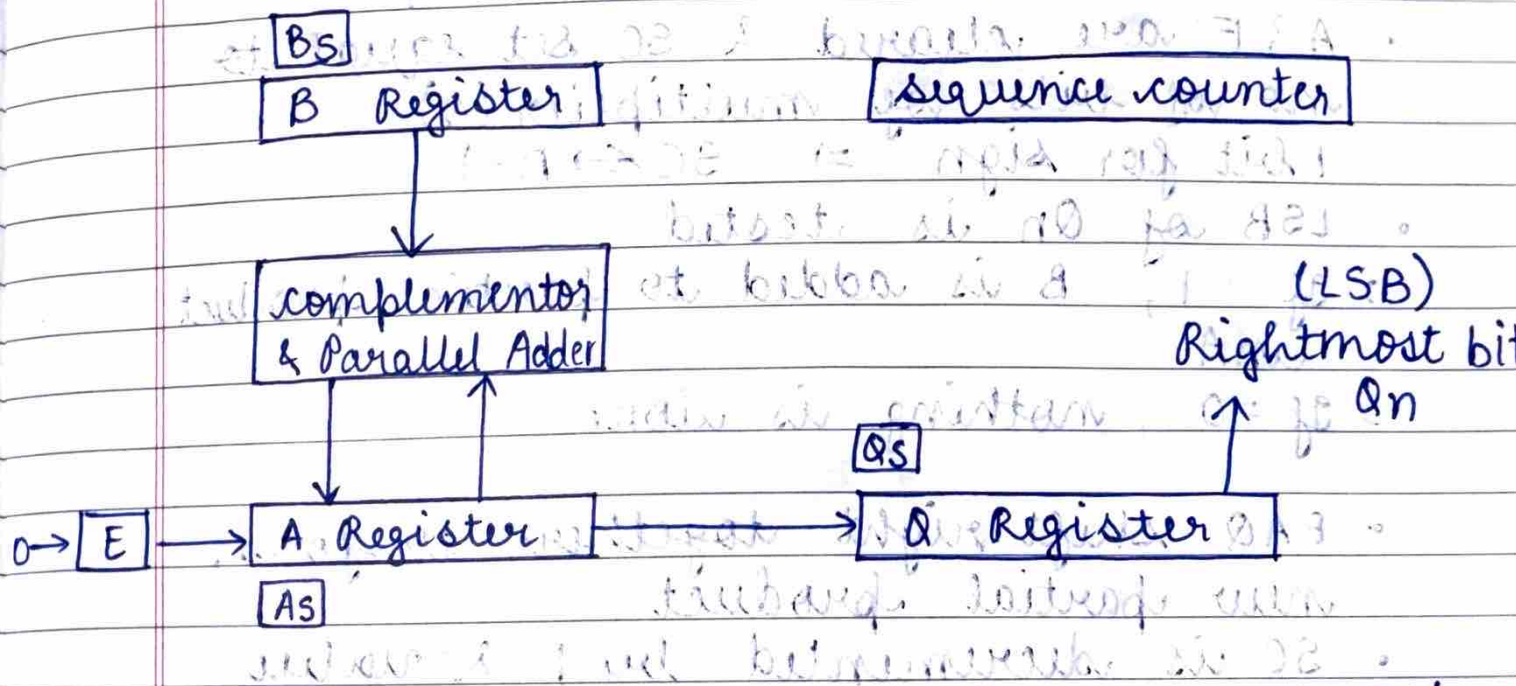
→ Multiplication Algorithms

1. Shift & Add operation

Just simple multiplication

eg→

```
  10111    (23) → MULTIPICAND
x 10011    (19) → MULTIPLIER
-------
  10111
 10111X
00000XX
00000XXX
10111XXXX
---------
110110101   (437)
```



B stores multiplicand Q → store multiplier

Bs → sign bit of B Qs → sign bit of Q

A → accumulator As → sign bit of A

E → stores Cout of A+B

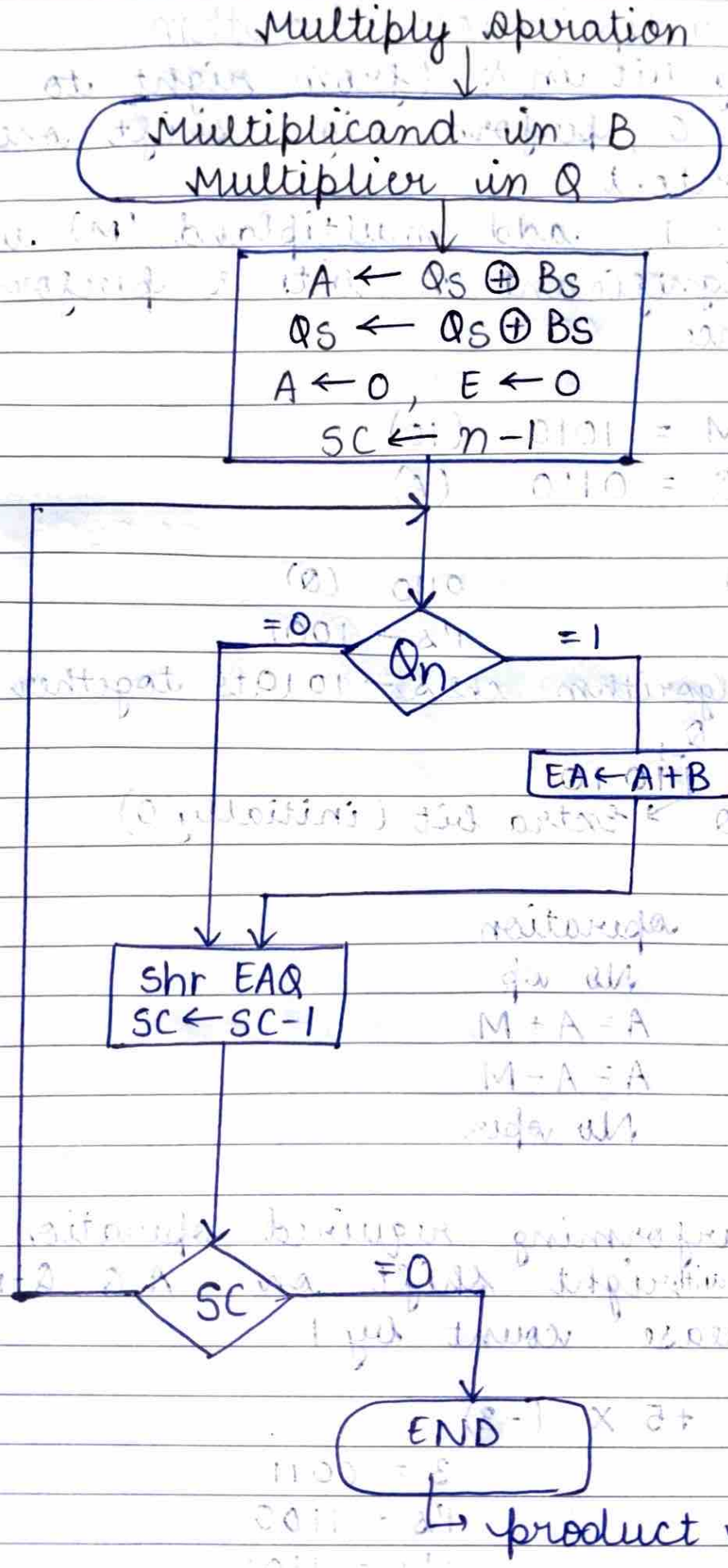
Sequence counter → counts steps left (N-1) initially.

Data Flow :-

1. A → adder
2. B → adder
3. adder → A (sum stored in A)
4. 0 → E (while shift)
5. E → A → Q all shift right

2. Hardware Algorithm

- Multiplicand stored in B & multiplier in Q, signs are Bs & Qs
- $B_s \oplus Q_s$ is found to detect sign of product ← add 1's detector
- $B_s \oplus Q_s$ is sent to both A & Q
- A & E are cleared & SC set equal to no of bits of multiplier (Q)
  1 bit for sign ⇒ SC ← n-1
- LSB of Qn is tested
  - if = 1, B is added to partial product in A
  - if = 0, nothing is done
- EAQ shift right together to form new partial product
- SC is decremented by 1 & value is checked if = 0, it stops else the process continues.
- Process stops when SC = 0.



FLOWCHART FOR HARDWARE ALGORITHM

↗ signed binary nos

☆☆ ↗ Done for nos in 2's complement notation

3. Booth's Multiplication Algorithm

- For every bit in Q (from right to left)

1. If bit is 0, perform right shift once on its content
2. If bit is 1, add multipland (M) with the most significant n bits & perform right shift once

eg → M = 1010 (10)
Q = 0110 (6)

1010 (M) 0110 (Q)

- Booth's algorithm checks 2 bits together
ie. Q_0 & Q_{-1}
↓ ↘ Extra bit (initially 0)
LSB of Q

Q_0	Q_{-1}	operation
0	0	No op
0	1	A = A + M
1	0	A = A - M
1	1	No oper

- After performing required operation always perform arithright shift on A, Q, Q_{-1} & decrease count by 1

eg :-
+5 X (-3)

5 = 0101 3 = 0011
1's = 1100
2's = 1101

Now

M = 0101 Q = 1101 count = 4 (no of bits)

A = 0000 Q-1 = 0

check Q_0 & Q-1 (1,0)

⇒ A = A - M

```
 0000
-0101
------
 1011
```

[→ 0101 → 2's = 1011
add ⇒ A-M = 0000 + 1011 = 1011]

Perform ashr of AQQ-1 together
(teeno ko ek sath likhke ashr krdo)

1011 1101 0

1 1011 1101 0

⇒ A = 1101 Q = 1110 & Q-1 = 1

c = 3

check Q, Q-1 = 01 → add A+M

A = A+M

= 1101 + 0101 = 10010

→ retain only 4 atmost bits

ashr

0010 1110

0001 0111 0

⇒ A = 0001 Q = 0111 Q-1 = 0 c = 2

Q, Q-1 = 10 A = A-M

M → 2's = 1011

A = A + 1011

```
 0001
 1011
------
 1100
```

⇒ A = 1100

ashr 1100 0111 0

1110 0011 1

A = 1110 Q = 0011 Q_{-1} = 1 C = 1

Q, Q_{-1} = 11

No change

ashr

1110 0011 1

11110011

A = 1111 Q = 0001 Q_{-1} = 1 C = 0

C = 0, not repeated

Final ans = AQ

= 11110001

↓

Negative number, find 2's complement

1's → 00001110

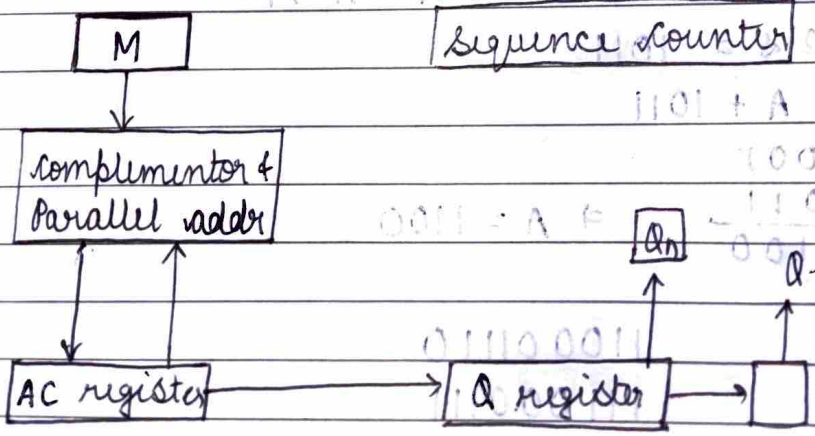
2's → 00001111

↳ magnitude = 15

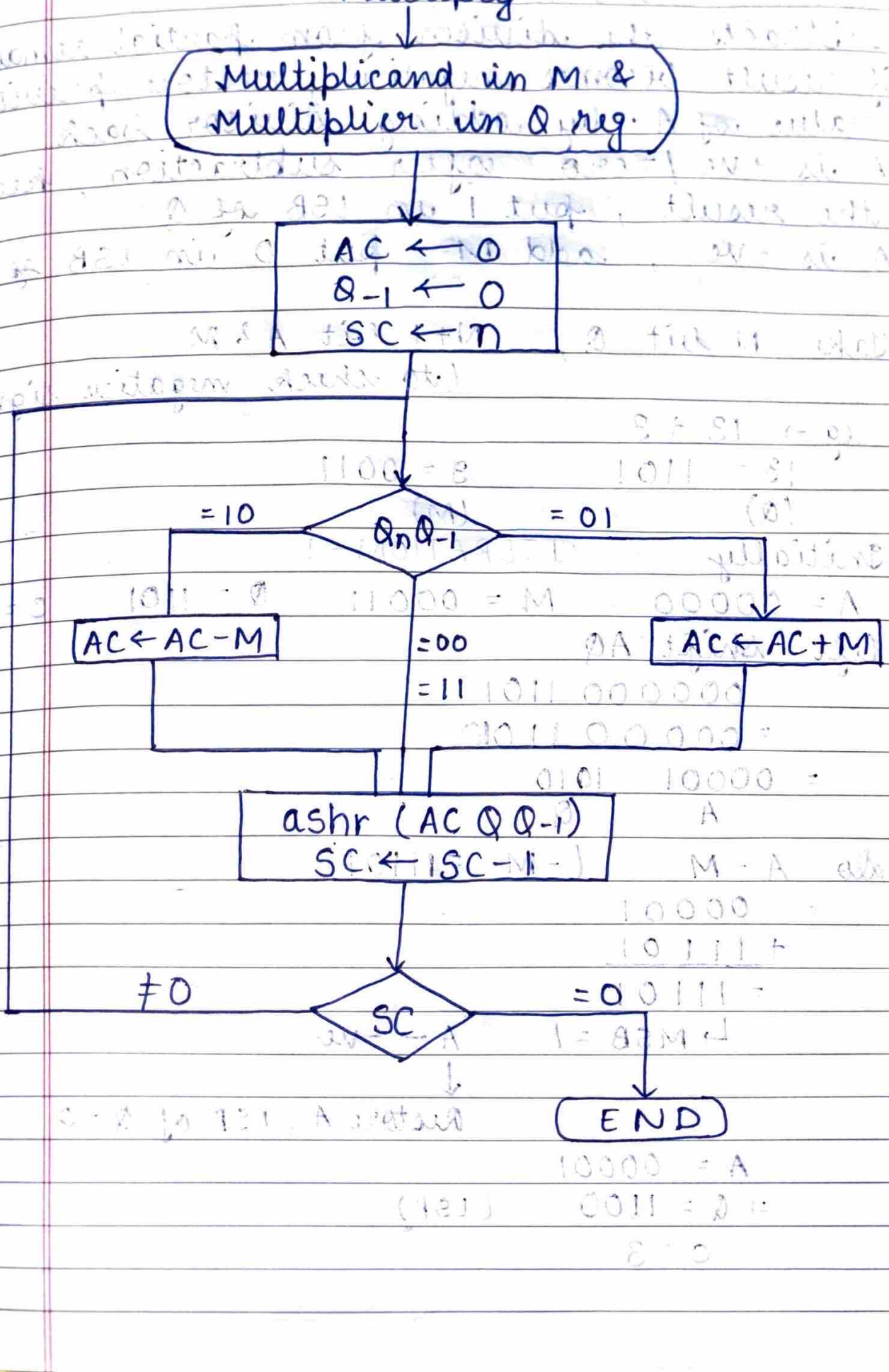
⇒ Ans = −15

→ Advantages

- Reduces number of arithematic operations when multiplier contains consecutive 1's
- Handles +ve & −ve numbers using same algorithm



Algorithm Flow chart



→ Division algorithms

1. Restoring division algorithm
   - Subtracts the divisor from partial remainder. If result becomes negative, restore previous value of A by adding divisor back.
   1. A is +ve / zero after subtraction, keep the result, put 1 in LSB of Q
   2. A is -ve, add M → put 0 in LSB of Q

- Take N bit Q, N+1 bit A & M (to check negative sign)

eg → 13 ÷ 3

13 = 1101 (Q) 3 = 0011 (M)

Initially ITERATION - 1

A = 00000 M = 00011 Q = 1101 C = 4

1. Left shift AQ

   00000 1101

   = 00001 1010

   = 00001 1010

   A Q

2. Do A - M (-M = 11101)

   = 00001

   + 11101

   = 11110

   ↳ MSB = 1 A → -ve

   ↓

   Restore A, LSB of Q = 0

   A = 00001

   ⇒ Q = 1010 (LSB)

   C = 3

ITERATION → 2

AQ = 00001 1010 —shl→ 000 1101 00

A = 00011 Q = 0100

A − M

00011

+ 11101

ignored ← (1)00000

only check N+1 bits

↳ +ve ⇒ $Q_0 = 1$

⇒ A = 00000 Q = 0101 C = 2

ITERATION → 3

AQ = 00000 0101 —shl→ 000001010

A = 00000 Q = 1010

A − M

00000

11101

11101

↳ −ve restore A, $Q_0 = 0$

11101

+ 00011

ignore ← (1)00000

⇒ A = 00000 Q = 1010 count = 1

ITERATION → 4

AQ = 00000 1010 —shl→ 000010100

A = 00001 Q = 0100

A − M 00001

+ 11101

11110

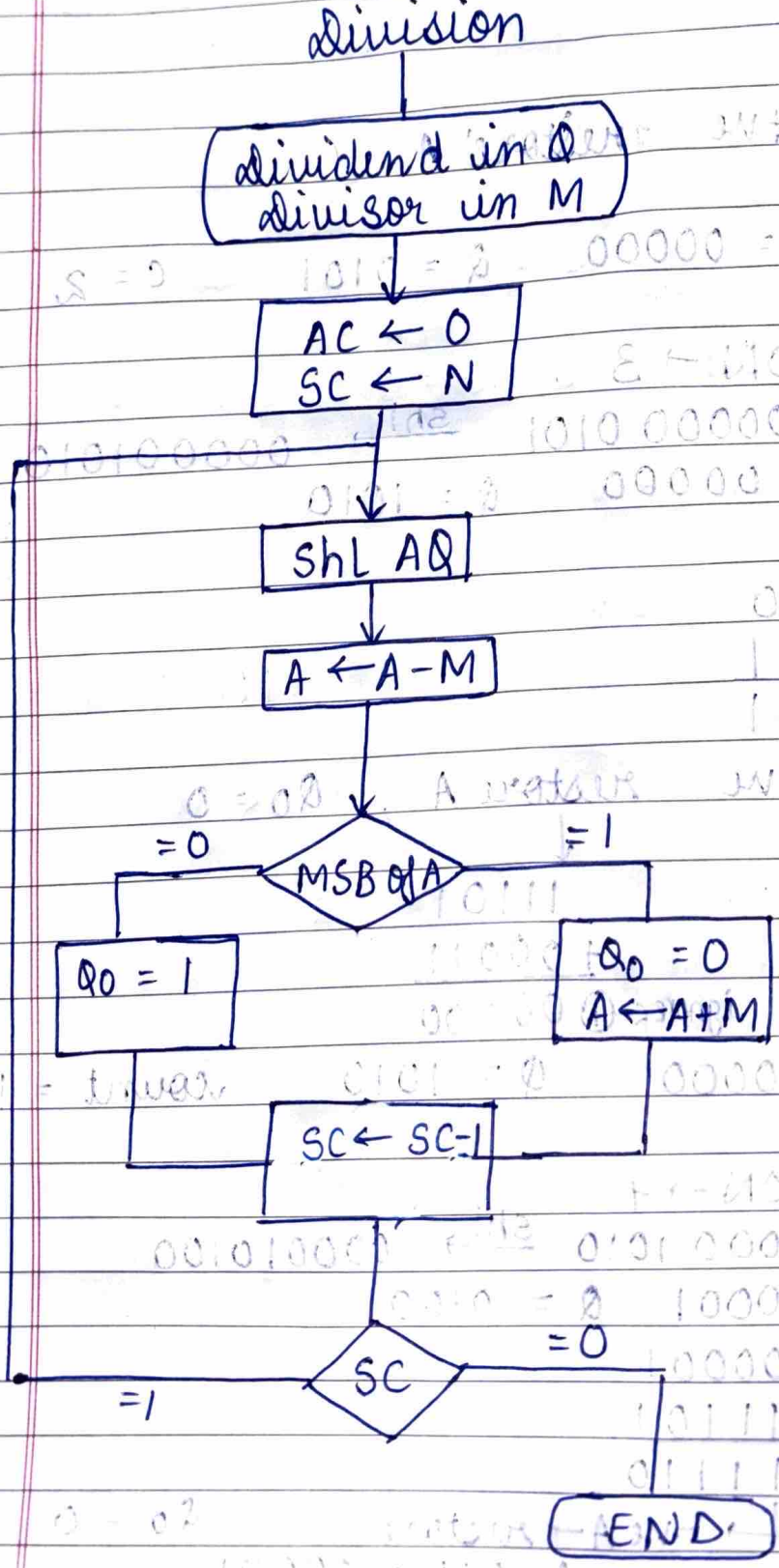
↳ +ve A → restore $Q_0 = 0$

A = A + M = 00001

A = 00001 Q = 0100 C = 0

Remainder ← A

Quotient ← Q



→ Non restoring Division Algorithm

- Performs similarly but doesn't restore A instead uses subtraction or addition depending upon previous sign.

Previous A	op after shl
A ≥ 0	A = A - M
A < 0	A = A + M

After op, if A = +ve, $Q_0 = 1$
A → -ve $Q_0 = 0$

After all iterations if A → negative
then A ← A + M

eg → 13 ÷ 3

A = 00000 Q = 1101 M = 00011 SC = 4

AQ = 00000 1101 —shl→ 00001 1010

A = 00001 Q = 1010

A - M = 00001 + 11101 = 11110

↳ A → -ve $Q_0 = 0$

A = 11110 Q = 1010 SC = 3

AQ = 11110 1010 —shl→ 11101 0100

A = 11101 Q = 0100

prev A → -ve do A + M

A + M = 11101 + 00011 = 00000

↳ A → +ve $Q_0 = 1$

A = 00000 Q = 0101 SC = 2

AQ = 00000 0101 —shl→ 00000 1010

A = 00000 Q = 1010

prev A → +ve do A - M

A - M = 00000 + 11101 = 11101

↳ -ve, $Q_0 = 0$

A = 11101 Q = 1010 SC = 1

AQ = 11101 1010 $\xrightarrow{shl}$ 11011 0100

A = 11011 Q = 0100

Brw A → -ve do A+M

A+M = 11011 + 00011

```
  11011
+ 00011
-------
  11110
```

↳ -ve $Q_0 = 0$

A = 11110 Q = 0100 SC = 0

Final correction, A → -ve ⇒ A ← A+M

```
  11110
+ 00011
-------
 100001
```

⇒ A = 00001

Finally,

A = 00001 Q = 0100

Remainder ← A ↳ Quotient

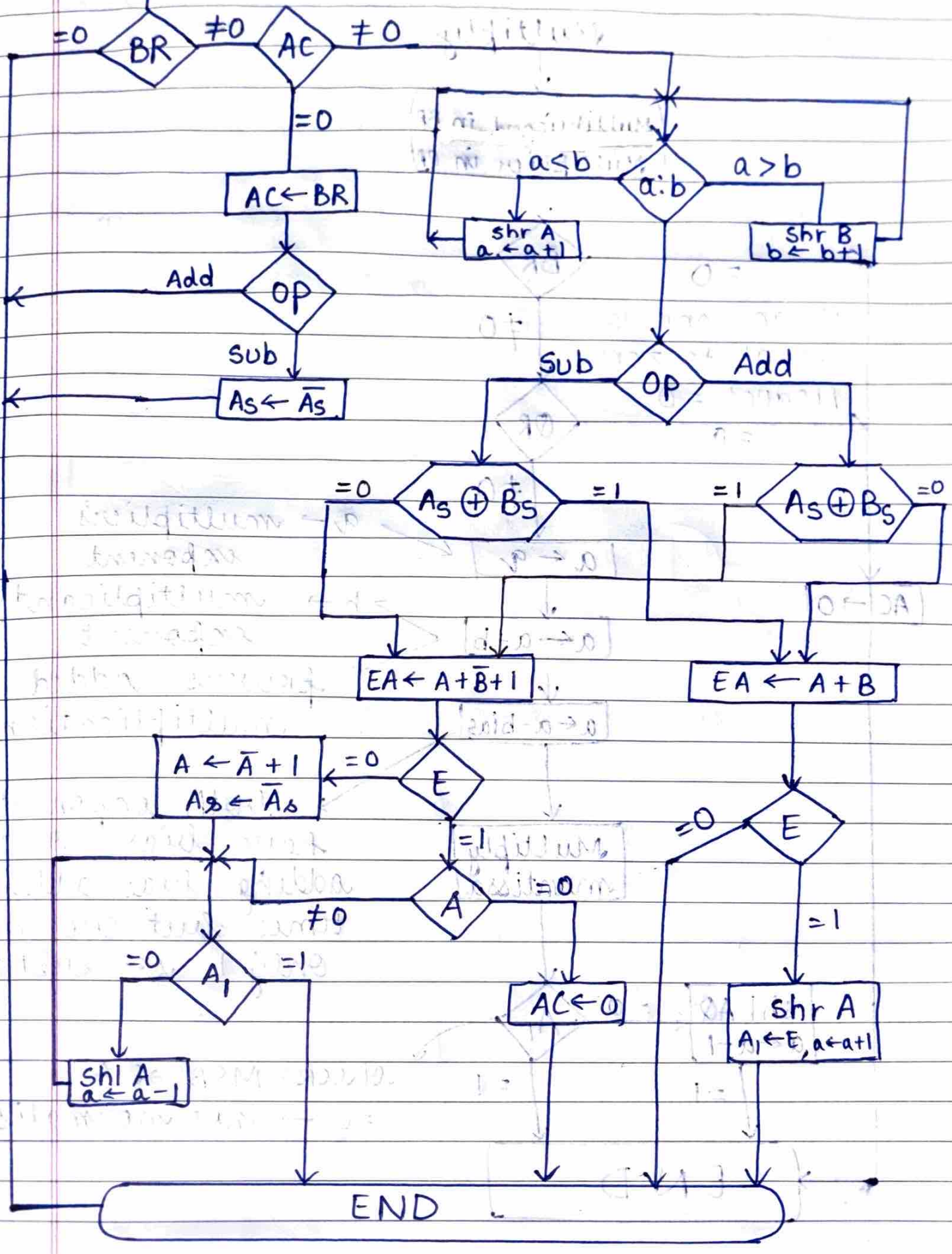
→ Floating pt add^n / subtraction

- 2 operands are stored in AC & BR
- sum / difference formed in AC

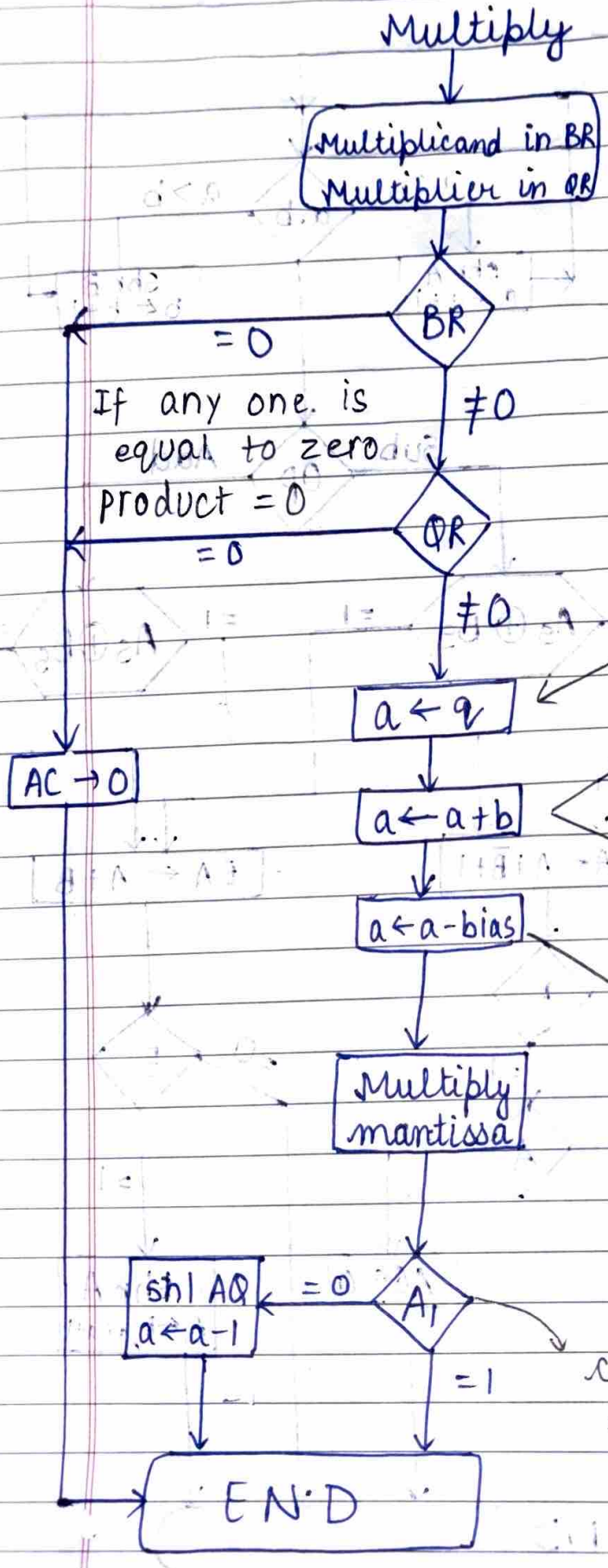
1. Check if = 0
2. Align mantissa
   - a < b → shr A, a ← a+1
   - a = b (aligned)
   - a > b → shr B, b ← b+1
3. Add / subtract mantissa
4. Normalise result

Add / subtract

E → carry bit



→ Multiplication of Floating point numbers



q → multiplier's exponent

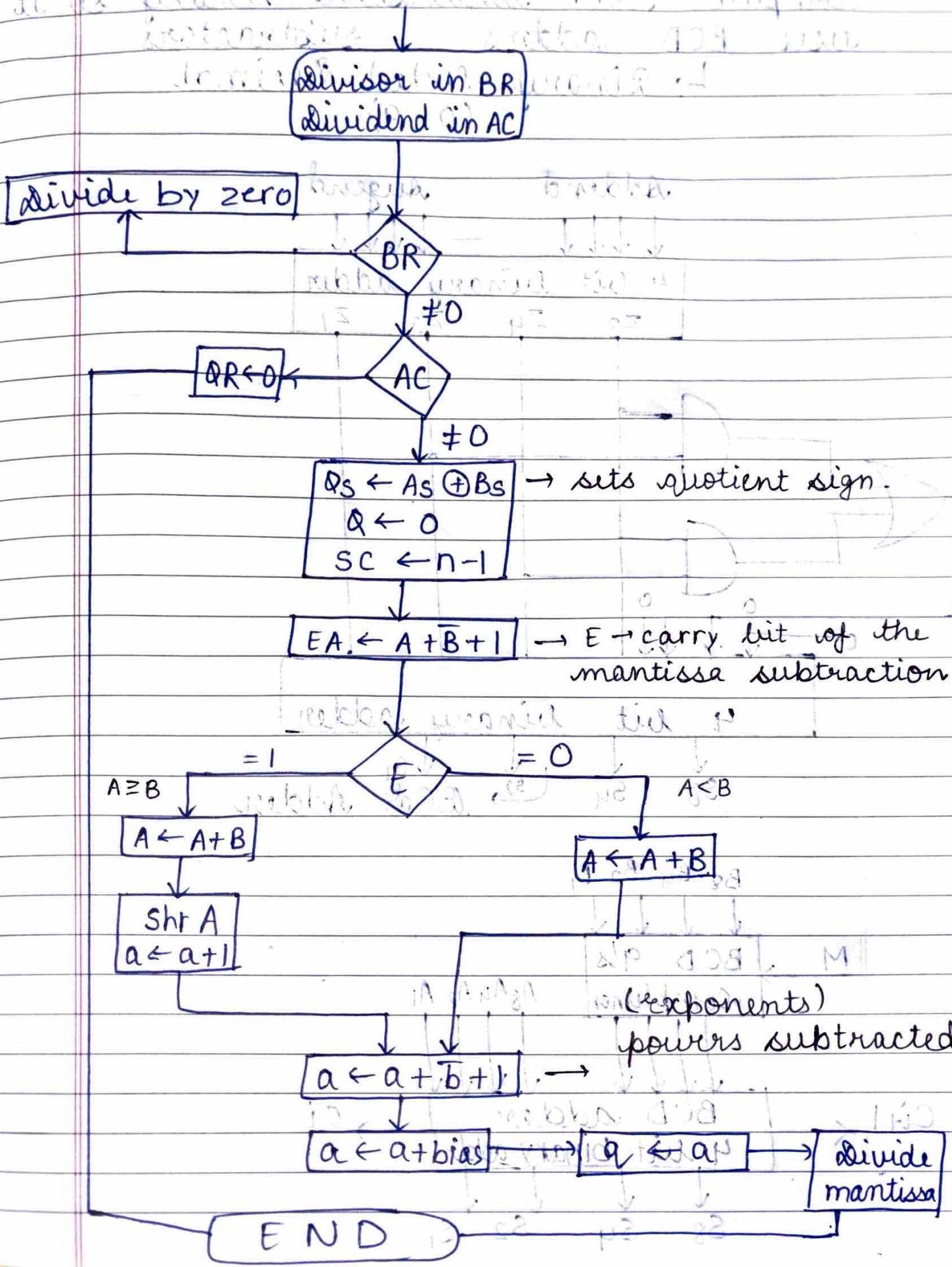
b → multiplicand's exponent

powers added in multiplication

both exponents have bias so after adding bias added 2 times but we need only 1 so subt one

checks MSB of A
=0 → not normalised

→ Division of Floating Point Numbers



→ Decimal arithematic units

Computer can't understand binary so uses BCD adders & subtractors

↳ Binary coded Decimal (0-9)

